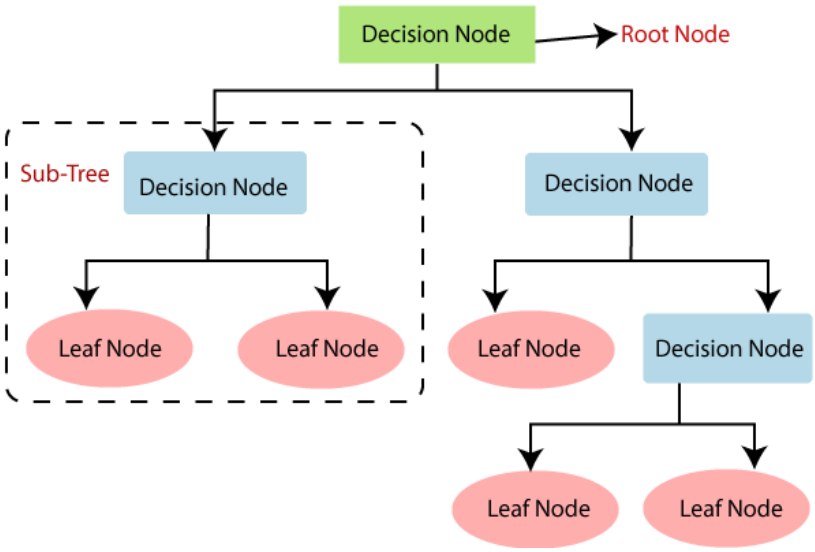
**UNIT - II Supervised Learning: Decision Tree Learning – Introduction, decision tree representation, appropriate problems for decision tree learning, the basic decision tree learning algorithm, Information Gain, Gini Index. issues in decision tree learning, overfitting, pruning.**

**Instance-Based Learning- Introduction, k-nearest neighbors' algorithm, remarks on lazy and eager learning**

Decision Tree Learning – Introduction, decision tree representation

- Decision Tree is a **Supervised learning technique** that can be used for **both classification and Regression problems**, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules** and **each leaf node represents the outcome.**
- In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node.** Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- The decisions or the test are performed on the basis of features of the given dataset.
- ***It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.***
- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.

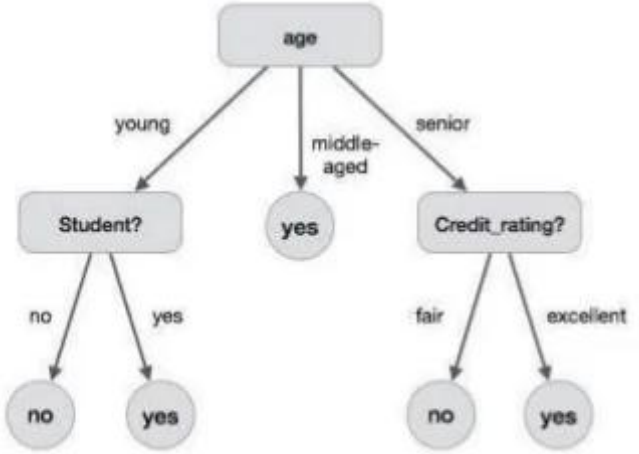



## Decision Tree Terminologies

- **Root Node:** Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.
- **Leaf Node:** Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.
- **Splitting:** Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.
- **Branch/Sub Tree:** A tree formed by splitting the tree.

- **Pruning:** Pruning is the process of removing the unwanted branches from the tree.
- **Parent/Child node:** The root node of the tree is called the parent node, and other nodes are called the child nodes.

The following decision tree is for the concept buy_computer that indicates whether a customer at a company is likely to buy a computer or not. Each internal node represents a test on an attribute. Each leaf node represents a class.




**Advantages**

Decision tree classification does not require any domain knowledge, hence, it is appropriate for the knowledge discovery process.

1. The representation of data in the form of the tree is easily understood by humans.
2. It can handle multidimensional data.
3. It is a quick process with great accuracy.

**Disadvantages**

1. Sometimes decision trees become very complex and these are called overfitted trees.
2. The decision tree algorithm may not be an optimal solution.
3. The decision trees may return a biased solution if some class label dominates it.

**Appropriate problems for Decision tree learning**

## APPROPRIATE PROBLEMS FOR DECISION TREE LEARNING

Decision tree learning is generally best suited to problems with the following characteristics:

1. **Instances are represented by attribute-value pairs.** Instances are described by a fixed set of attributes (e.g., **Temperature)** and their values (e.g., **Hot).** The easiest situation for decision tree learning is when each attribute takes on a small number of disjoint possible values (e.g., **Hot, Mild, Cold).**

**Algorithm:**

A machine researcher named J. Ross Quinlan in 1980 developed a decision tree algorithm known as ID3 (Iterative Dichotomiser). Later, he presented C4.5, which was the successor of ID3. ID3 and C4.5 adopt a greedy approach. In this algorithm, there is no backtracking; the trees are constructed in a top-down recursive divide-and-conquer manner.

Generating a decision tree from training tuples of data partition D

Algorithm : Generate_decision_tree

Input:

Data partition, D, which is a set of training tuples and their associated class labels.

attribute_list, the set of candidate attributes. Attribute selection method, a procedure to determine the splitting criterion that best partitions that the data tuples into individual classes. This criterion includes a splitting_attribute and either a splitting point or splitting subset.

Output:

A Decision Tree

Method

create a node N;

if tuples in D are all of the same class, C then

return N as leaf node labeled with class C;

if attribute_list is empty then

return N as leaf node with labeled

with majority class in D;|| majority voting

apply attribute_selection_method(D, attribute_list)

to find the best splitting_criterion;

label node N with splitting_criterion;

if splitting_attribute is discrete-valued and

multiway splits allowed then // no restricted to binary trees

attribute_list = splitting attribute; // remove splitting attribute

for each outcome j of splitting criterion

// partition the tuples and grow subtrees for each partition

let Dj be the set of data tuples in D satisfying outcome j; // a partition

if Dj is empty then

attach a leaf labeled with the majority

class in D to node N;

else

attach the node returned by Generate

decision tree(Dj, attribute list) to node N;

end for

return N;

Example:

| Day | Outlook | Temp | Humidity | Wind | PlayTennis |
|---|---|---|---|---|---|
| D1 | Sunny | Hot | High | Weak | No |
| D2 | Sunny | Hot | High | Strong | No |
| D3 | Overcast | Hot | High | Weak | Yes |
| D4 | Rain | Mild | High | Weak | Yes |
| D5 | Rain | Cool | Normal | Weak | Yes |
| D6 | Rain | Cool | Normal | Strong | No |
| D7 | Overcast | Cool | Normal | Strong | Yes |
| D8 | Sunny | Mild | High | Weak | No |
| D9 | Sunny | Cool | Normal | Weak | Yes |
| D10 | Rain | Mild | Normal | Weak | Yes |
| D11 | Sunny | Mild | Normal | Strong | Yes |
| D12 | Overcast | Mild | High | Strong | Yes |
| D13 | Overcast | Hot | Normal | Weak | Yes |
| D14 | Rain | Mild | High | Strong | No |

We have to identify which attribute is giving maximum information from the given attribute.

From the above example (outlook, temp, humidity wind)

**step1:** find information gain for each attribute.

To find **information gain** of an attribute first we have to find **ENTROPY** of whole dataset and the entropy of individual values of the attribute

*Entropy is the measurement of impurities or randomness in the data points.*

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

| Day | Outlook | Temp | Humidity | Wind | Play Tennis |
|---|---|---|---|---|---|
| D1 | Sunny | Hot | High | Weak | No |
| D2 | Sunny | Hot | High | Strong | No |
| D3 | Overcast | Hot | High | Weak | Yes |
| D4 | Rain | Mild | High | Weak | Yes |
| D5 | Rain | Cool | Normal | Weak | Yes |
| D6 | Rain | Cool | Normal | Strong | No |
| D7 | Overcast | Cool | Normal | Strong | Yes |
| D8 | Sunny | Mild | High | Weak | No |
| D9 | Sunny | Cool | Normal | Weak | Yes |
| D10 | Rain | Mild | Normal | Weak | Yes |
| D11 | Sunny | Mild | Normal | Strong | Yes |
| D12 | Overcast | Mild | High | Strong | Yes |
| D13 | Overcast | Hot | Normal | Weak | Yes |
| D14 | Rain | Mild | High | Strong | No |

**Attribute: Outlook**

$Values(Outlook) = Sunny, Overcast, Rain$

$S = [9+, 5-]$ $\quad Entropy(S) = -\frac{9}{14}log_2\frac{9}{14} - \frac{5}{14}log_2\frac{5}{14} = 0.94$

$S_{Sunny} \leftarrow [2+, 3-]$ $\quad Entropy(S_{Sunny}) = -\frac{2}{5}log_2\frac{2}{5} - \frac{3}{5}log_2\frac{3}{5} = 0.971$

$S_{Overcast} \leftarrow [4+, 0-]$ $\quad Entropy(S_{Overcast}) = -\frac{4}{4}log_2\frac{4}{4} - \frac{0}{4}log_2\frac{0}{4} = 0$

$S_{Rain} \leftarrow [3+, 2-]$ $\quad Entropy(S_{Rain}) = -\frac{3}{5}log_2\frac{3}{5} - \frac{2}{5}log_2\frac{2}{5} = 0.971$

$$Gain\,(S, Outlook) = Entropy(S) - \sum_{v \in \{Sunny, Overcast, Rain\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$Gain(S, Outlook)$

$$= Entropy(S) - \frac{5}{14}Entropy(S_{Sunny}) - \frac{4}{14}Entropy(S_{Overcast}) - \frac{5}{14}Entropy(S_{Rain})$$

$$Gain(S, Outlook) = 0.94 - \frac{5}{14}0.971 - \frac{4}{14}0 - \frac{5}{14}0.971 = 0.2464$$

Step 2:

| Day | Outlook | Temp | Humidity | Wind | Play Tennis |
|---|---|---|---|---|---|
| D1 | Sunny | Hot | High | Weak | No |
| D2 | Sunny | Hot | High | Strong | No |
| D3 | Overcast | Hot | High | Weak | Yes |
| D4 | Rain | Mild | High | Weak | Yes |
| D5 | Rain | Cool | Normal | Weak | Yes |
| D6 | Rain | Cool | Normal | Strong | No |
| D7 | Overcast | Cool | Normal | Strong | Yes |
| D8 | Sunny | Mild | High | Weak | No |
| D9 | Sunny | Cool | Normal | Weak | Yes |
| D10 | Rain | Mild | Normal | Weak | Yes |
| D11 | Sunny | Mild | Normal | Strong | Yes |
| D12 | Overcast | Mild | High | Strong | Yes |
| D13 | Overcast | Hot | Normal | Weak | Yes |
| D14 | Rain | Mild | High | Strong | No |

**Attribute: Temp**

$Values\ (Temp) = Hot, Mild, Cool$

$S = [9+, 5-]$ $\quad Entropy(S) = -\frac{9}{14}log_2\frac{9}{14} - \frac{5}{14}log_2\frac{5}{14} = 0.94$

$S_{Hot} \leftarrow [2+, 2-]$ $\quad Entropy(S_{Hot}) = -\frac{2}{4}log_2\frac{2}{4} - \frac{2}{4}log_2\frac{2}{4} = 1.0$

$S_{Mild} \leftarrow [4+, 2-]$ $\quad Entropy(S_{Mild}) = -\frac{4}{6}log_2\frac{4}{6} - \frac{2}{6}log_2\frac{2}{6} = 0.9183$

$S_{Cool} \leftarrow [3+, 1-]$ $\quad Entropy(S_{Cool}) = -\frac{3}{4}log_2\frac{3}{4} - \frac{1}{4}log_2\frac{1}{4} = 0.8113$

$$Gain\ (S, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Temp) = Entropy(S) - \frac{4}{14}Entropy(S_{Hot}) - \frac{6}{14}Entropy(S_{Mild}) - \frac{4}{14}Entropy(S_{Cool})$$

$$Gain(S, Temp) = 0.94 - \frac{4}{14}1.0 - \frac{6}{14}0.9183 - \frac{4}{14}0.8113 = 0.0289$$

vtupulse.com

Step 3:

| Day | Outlook | Temp | Humidity | Wind | Play Tennis |
|---|---|---|---|---|---|
| D1 | Sunny | Hot | High | Weak | No |
| D2 | Sunny | Hot | High | Strong | No |
| D3 | Overcast | Hot | High | Weak | Yes |
| D4 | Rain | Mild | High | Weak | Yes |
| D5 | Rain | Cool | Normal | Weak | Yes |
| D6 | Rain | Cool | Normal | Strong | No |
| D7 | Overcast | Cool | Normal | Strong | Yes |
| D8 | Sunny | Mild | High | Weak | No |
| D9 | Sunny | Cool | Normal | Weak | Yes |
| D10 | Rain | Mild | Normal | Weak | Yes |
| D11 | Sunny | Mild | Normal | Strong | Yes |
| D12 | Overcast | Mild | High | Strong | Yes |
| D13 | Overcast | Hot | Normal | Weak | Yes |
| D14 | Rain | Mild | High | Strong | No |

**Attribute: Humidity**

$Values\ (Humidity) = High, Normal$

$S = [9+, 5-]$ $\quad Entropy(S) = -\frac{9}{14}log_2\frac{9}{14} - \frac{5}{14}log_2\frac{5}{14} = 0.94$

$S_{High} \leftarrow [3+, 4-]$ $\quad Entropy(S_{High}) = -\frac{3}{7}log_2\frac{3}{7} - \frac{4}{7}log_2\frac{4}{7} = 0.9852$

$S_{Normal} \leftarrow [6+, 1-]$ $\quad Entropy(S_{Normal}) = -\frac{6}{7}log_2\frac{6}{7} - \frac{1}{7}log_2\frac{1}{7} = 0.5916$

$$Gain\ (S, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Humidity) = Entropy(S) - \frac{7}{14}Entropy(S_{High}) - \frac{7}{14}Entropy(S_{Normal})$$

$$Gain(S, Humidity) = 0.94 - \frac{7}{14}0.9852 - \frac{7}{14}0.5916 = 0.1516$$

Step4:

| Day | Outlook | Temp | Humidity | Wind | Play Tennis |
|---|---|---|---|---|---|
| D1 | Sunny | Hot | High | Weak | No |
| D2 | Sunny | Hot | High | Strong | No |
| D3 | Overcast | Hot | High | Weak | Yes |
| D4 | Rain | Mild | High | Weak | Yes |
| D5 | Rain | Cool | Normal | Weak | Yes |
| D6 | Rain | Cool | Normal | Strong | No |
| D7 | Overcast | Cool | Normal | Strong | Yes |
| D8 | Sunny | Mild | High | Weak | No |
| D9 | Sunny | Cool | Normal | Weak | Yes |
| D10 | Rain | Mild | Normal | Weak | Yes |
| D11 | Sunny | Mild | Normal | Strong | Yes |
| D12 | Overcast | Mild | High | Strong | Yes |
| D13 | Overcast | Hot | Normal | Weak | Yes |
| D14 | Rain | Mild | High | Strong | No |

**Attribute: Wind**

$Values(Wind) = Strong, Weak$

$S = [9+, 5-]$ $\quad Entropy(S) = -\frac{9}{14}log_2\frac{9}{14} - \frac{5}{14}log_2\frac{5}{14} = 0.94$

$S_{Strong} \leftarrow [3+, 3-]$ $\quad Entropy(S_{Strong}) = 1.0$

$S_{Weak} \leftarrow [6+, 2-]$ $\quad Entropy(S_{Weak}) = -\frac{6}{8}log_2\frac{6}{8} - \frac{2}{8}log_2\frac{2}{8} = 0.8113$

$$Gain(S, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Wind) = Entropy(S) - \frac{6}{14}Entropy(S_{Strong}) - \frac{8}{14}Entropy(S_{Weak})$$

$$Gain(S, Wind) = 0.94 - \frac{6}{14}1.0 - \frac{8}{14}0.8113 = 0.0478$$

Step5:

| Day | Outlook | Temp | Humidity | Wind | Play Tennis |
|---|---|---|---|---|---|
| D1 | Sunny | Hot | High | Weak | No |
| D2 | Sunny | Hot | High | Strong | No |
| D3 | Overcast | Hot | High | Weak | Yes |
| D4 | Rain | Mild | High | Weak | Yes |
| D5 | Rain | Cool | Normal | Weak | Yes |
| D6 | Rain | Cool | Normal | Strong | No |
| D7 | Overcast | Cool | Normal | Strong | Yes |
| D8 | Sunny | Mild | High | Weak | No |
| D9 | Sunny | Cool | Normal | Weak | Yes |
| D10 | Rain | Mild | Normal | Weak | Yes |
| D11 | Sunny | Mild | Normal | Strong | Yes |
| D12 | Overcast | Mild | High | Strong | Yes |
| D13 | Overcast | Hot | Normal | Weak | Yes |
| D14 | Rain | Mild | High | Strong | No |

$$Gain(S, Outlook) = 0.2464$$

$$Gain(S, Temp) = 0.0289$$

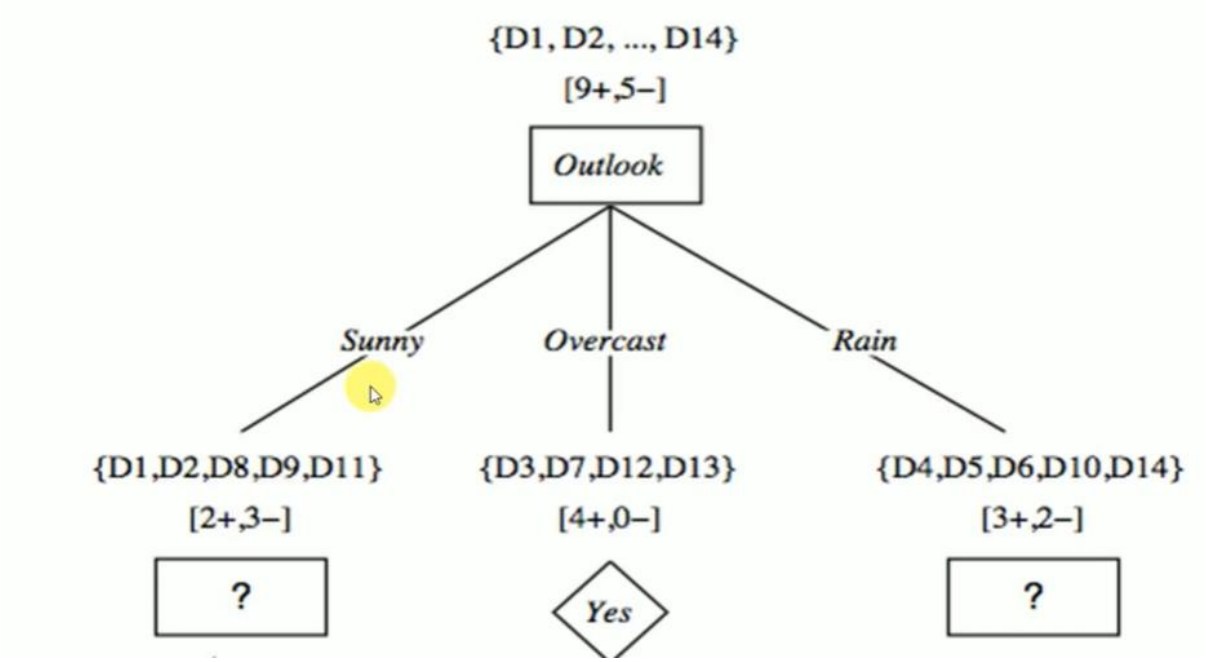
$$Gain(S, Humidity) = 0.1516$$

$$Gain(S, Wind) = 0.0478$$

Here outlook is having highest gain (i. 0.2464) so take this attribute as root node.

Now construct a tree

Step 6:




Yes is the leaf node for overcast

As we have a combination of Yes and NO i.e for example D1,D2,D8,D9,D11 has 2 Yes and 3 No so we should not label it . In the same way with D4,D5,D6,D10,D14

Step 7:

Now we should not consider outlook attribute

| Day | Temp | Humidity | Wind | Play Tennis |
|---|---|---|---|---|
| D1 | Hot | High | Weak | No |
| D2 | Hot | High | Strong | No |
| D8 | Mild | High | Weak | No |
| D9 | Cool | Normal | Weak | Yes |
| D11 | Mild | Normal | Strong | Yes |

## Attribute: Temp

$Values\ (Temp) = Hot, Mild, Cool$

$S_{Sunny} = [2+, 3-]$ $\quad Entropy(S_{Sunny}) = -\frac{2}{5}log_2\frac{2}{5} - \frac{3}{5}log_2\frac{3}{5} = 0.97$

$S_{Hot} \leftarrow [0+, 2-]$ $\quad Entropy(S_{Hot}) = 0.0$

$S_{Mild} \leftarrow [1+, 1-]$ $\quad Entropy(S_{Mild}) = 1.0$

$S_{Cool} \leftarrow [1+, 0-]$ $\quad Entropy(S_{Cool}) = 0.0$

$$Gain\ (S_{Sunny}, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Sunny}, Temp)$$

$$= Entropy(S) - \frac{2}{5}Entropy(S_{Hot}) - \frac{2}{5}Entropy(S_{Mild})$$

$$- \frac{1}{5}Entropy(S_{Cool})$$

$$Gain(S_{sunny}, Temp) = 0.97 - \frac{2}{5}0.0 - \frac{2}{5}1 - \frac{1}{5}0.0 = 0.570$$

## Attribute: Humidity

$Values\ (Humidity) = High, Normal$

$S_{Sunny} = [2+, 3-]$ $\quad Entropy(S) = -\frac{2}{5}log_2\frac{2}{5} - \frac{3}{5}log_2\frac{3}{5} = 0.97$

$S_{high} \leftarrow [0+, 3-]$ $\quad Entropy(S_{High}) = 0.0$

$S_{Normal} \leftarrow [2+, 0-]$ $\quad Entropy(S_{Normal}) = 0.0$

$$Gain\ (S_{Sunny}, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Sunny}, Humidity) = Entropy(S) - \frac{3}{5}Entropy(S_{High}) - \frac{2}{5}Entropy(S_{Normal})$$

$$Gain(S_{sunny}, Humidity) = 0.97 - \frac{3}{5}0.0 - \frac{2}{5}0.0 = 0.97$$

## Attribute: Wind

$Values\ (Wind) = Strong, Weak$

$S_{Sunny} = [2+, 3-]$ $\quad Entropy(S) = -\frac{2}{5} log_2 \frac{2}{5} - \frac{3}{5} log_2 \frac{3}{5} = 0.97$

$S_{Strong} \leftarrow [1+, 1-]$ $\quad Entropy(S_{Strong}) = 1.0$

$S_{Weak} \leftarrow [1+, 2-]$ $\quad Entropy(S_{Weak}) = -\frac{1}{3} log_2 \frac{1}{3} - \frac{2}{3} log_2 \frac{2}{3} = 0.9183$

$$Gain\ (S_{Sunny}, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

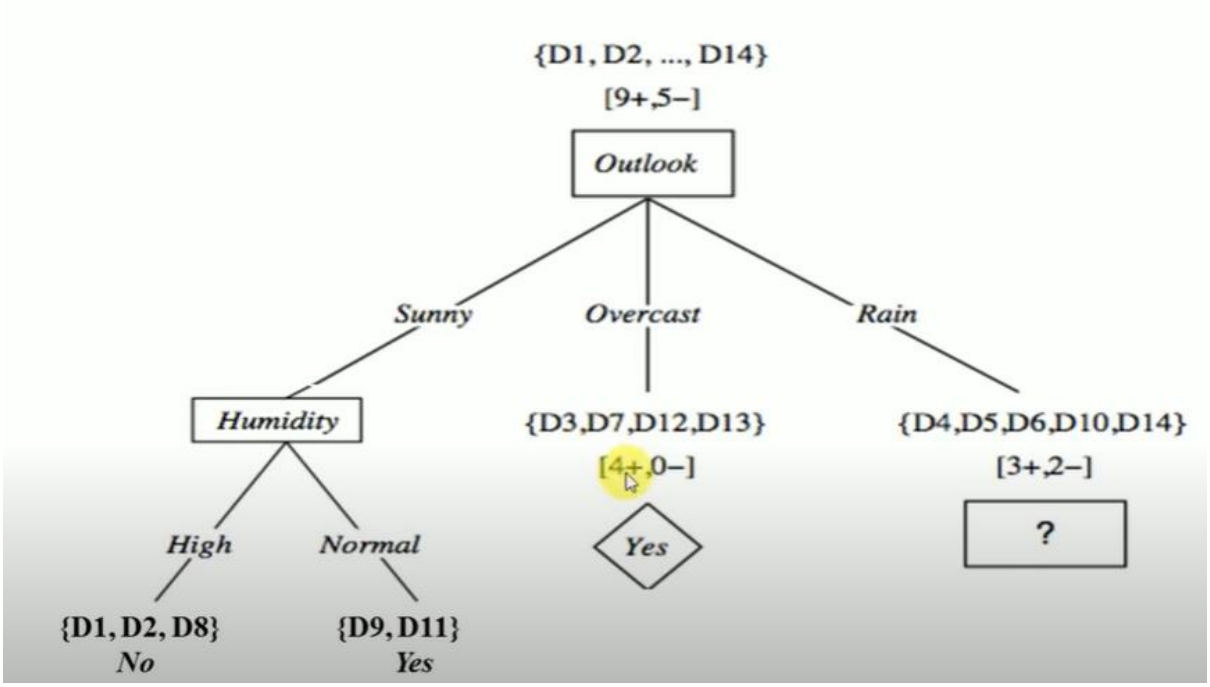
$$Gain(S_{Sunny}, Wind) = Entropy(S) - \frac{2}{5} Entropy(S_{Strong}) - \frac{3}{5} Entropy(S_{Weak})$$

$$Gain(S_{sunny}, Wind) = 0.97 - \frac{2}{5} 1.0 - \frac{3}{5}\ 0.918 = 0.0192$$

$$Gain(S_{sunny}, Temp) = 0.570$$

$$Gain(S_{sunny}, Humidity) = 0.97$$

$$Gain(S_{sunny}, Wind) = 0.0192$$




Step7:

| Day | Temp | Humidity | Wind | Play Tennis |
|---|---|---|---|---|
| D4 | Mild | High | Weak | Yes |
| D5 | Cool | Normal | Weak | Yes |
| D6 | Cool | Normal | Strong | No |
| D10 | Mild | Normal | Weak | Yes |
| D14 | Mild | High | Strong | No |

**Attribute: Temp**

$Values\ (Temp) = Hot, Mild, Cool$

$S_{Rain} = [3+, 2-]$ $\quad Entropy(S_{Sunny}) = -\frac{3}{5}log_2\frac{3}{5} - \frac{2}{5}log_2\frac{2}{5} = 0.97$

$S_{Hot} \leftarrow [0+, 0-]$ $\quad Entropy(S_{Hot}) = 0.0$

$S_{Mild} \leftarrow [2+, 1-]$ $\quad Entropy(S_{Mild}) = -\frac{2}{3}log_2\frac{2}{3} - \frac{1}{3}log_2\frac{1}{3} = 0.9183$

$S_{Cool} \leftarrow [1+, 1-]$ $\quad Entropy(S_{Cool}) = 1.0$

$$Gain\ (S_{Rain}, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Rain}, Temp)$$

$$= Entropy(S) - \frac{0}{5}Entropy(S_{Hot}) - \frac{3}{5}Entropy(S_{Mild})$$

$$- \frac{2}{5}Entropy(S_{Cool})$$

$$Gain(S_{Rain}, Temp) = 0.97 - \frac{0}{5}0.0 - \frac{3}{5}0.918 - \frac{2}{5}1.0 = 0.0192$$

## Attribute: Humidity

$Values\ (Humidity) = High, Normal$

$S_{Rain} = [3+, 2-]$ $\quad Entropy(S_{Sunny}) = -\frac{3}{5}log_2\frac{3}{5} - \frac{2}{5}log_2\frac{2}{5} = 0.97$

$S_{High} \leftarrow [1+, 1-]$ $\quad Entropy(S_{High}) = 1.0$

$S_{Normal} \leftarrow [2+, 1-]$ $\quad Entropy(S_{Normal}) = -\frac{2}{3}log_2\frac{2}{3} - \frac{1}{3}log_2\frac{1}{3} = 0.9183$

$$Gain\ (S_{Rain}, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Rain}, Humidity) = Entropy(S) - \frac{2}{5}Entropy(S_{High}) - \frac{3}{5}Entropy(S_{Normal})$$

$$Gain(S_{Rain}, Humidity) = 0.97 - \frac{2}{5}\ 1.0 - \frac{3}{5}0.918 = 0.0192$$

## Attribute: Wind

$Values\ (wind) = Strong, Weak$

$S_{Rain} = [3+, 2-]$ $\quad Entropy(S_{Sunny}) = -\frac{3}{5}log_2\frac{3}{5} - \frac{2}{5}log_2\frac{2}{5} = 0.97$

$S_{Strong} \leftarrow [0+, 2-]$ $\quad Entropy(S_{Strong}) = 0.0$

$S_{Weak} \leftarrow [3+, 0-]$ $\quad Entropy(S_{weak}) = 0.0$

$$Gain\ (S_{Rain}, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

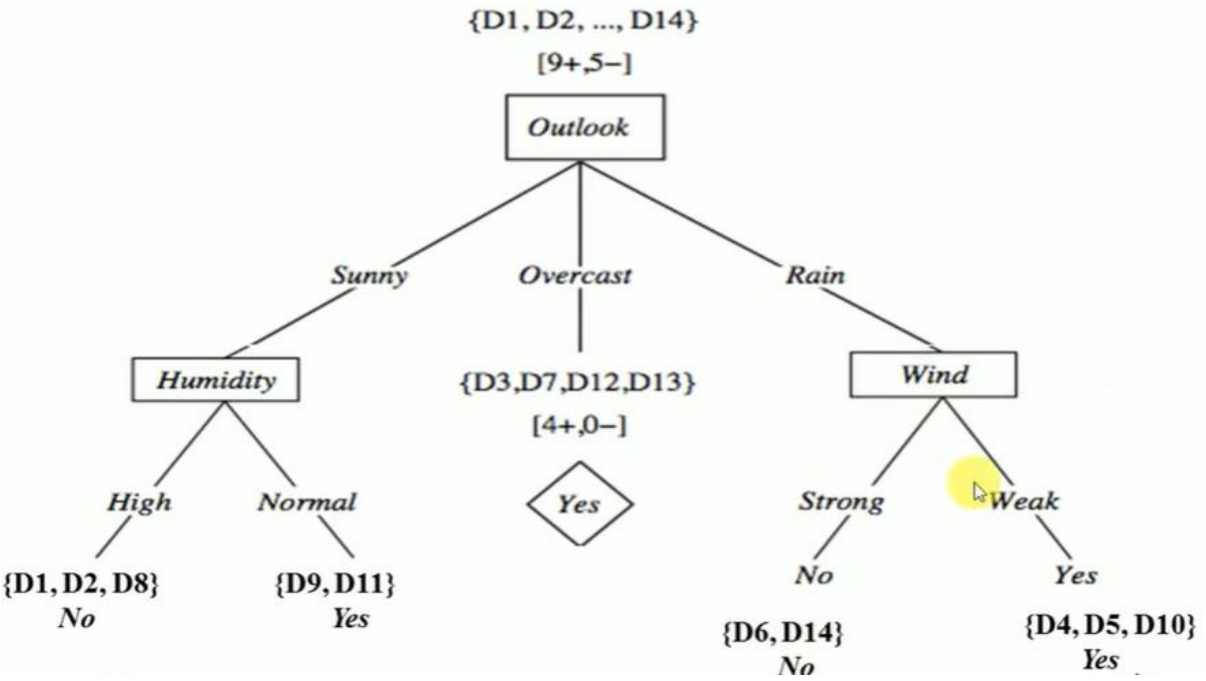
$$Gain(S_{Rain}, Wind) = Entropy(S) - \frac{2}{5}Entropy(S_{Strong}) - \frac{3}{5}Entropy(S_{Weak})$$

$$Gain(S_{Rain}, Wind) = 0.97 - \frac{2}{5}\ 0.0 - \frac{3}{5}0.0 = 0.97$$

$$Gain(S_{Rain}, Temp) = 0.0192$$

$$Gain(S_{Rain}, Humidity) = 0.0192$$

$$Gain(S_{Rain}, Wind) = 0.97$$




**Note: ID3 algorithm attributes with large number of values, bias for multivalued attributes**

## Gain Ratio

**C4.5 is a Successor of ID3 algorithm**

- Improved version of ID3
  - Gain Ratio:

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo(A)}$$

$$SplitInfo_A(D) = -\sum_{j=1}^{v} \frac{|D_j|}{|D|} \times \log_2\left(\frac{|D_j|}{|D|}\right)$$

**$D_j$ is Number of samples in a dataset for an particular outcome j of an attribute**

**D – total number of samples in a dateset.**

- C4.5 uses gain ratio to overcome the problem

**Ex:**

| CGPA | Inter active | Practical Knowledge | Comm Skills | Job Offer |
|---|---|---|---|---|
| >=9 | Yes | Very good | Good | Yes |
| >=8 | No | Good | Moderate | Yes |
| >=9 | No | Average | Poor | No |
| <8 | No | Average | Good | No |
| >=8 | Yes | Good | Moderate | Yes |
| >=9 | Yes | Good | Moderate | Yes |
| <8 | Yes | Good | Poor | No |
| >=9 | No | Very good | Good | Yes |
| >=8 | Yes | Good | Good | Yes |
| >=8 | Yes | Average | Good | Yes |

$$\text{Entropy_Info}(T) = -\sum_{i=1}^{m} P_i \log_2 P_i$$

$$\text{Entropy_Info}(\text{Target Attribute} = \text{Job Offer}) = \text{Entropy_Info}(7, 3) =$$
$$= -\left[\frac{7}{10}\log_2\frac{7}{10} + \frac{3}{10}\log_2\frac{3}{10}\right]$$
$$= (-0.3599 + -0.5208)$$
$$= 0.8807$$

**Ex:**

| age | income | student | credit_rating | buys_computer |
|---|---|---|---|---|
| youth | high | no | fair | no |
| youth | high | no | excellent | no |
| middle_aged | high | no | fair | yes |
| senior | medium | no | fair | yes |
| senior | low | yes | fair | yes |
| senior | low | yes | excellent | no |
| middle_aged | low | yes | excellent | yes |
| youth | medium | no | fair | no |
| youth | low | yes | fair | yes |
| senior | medium | yes | fair | yes |
| youth | medium | yes | excellent | yes |
| middle_aged | medium | no | excellent | yes |
| middle_aged | high | yes | fair | yes |
| senior | medium | no | excellent | no |

$$SplitInfo_{income}(D) = -\frac{4}{14} \times \log_2\left(\frac{4}{14}\right) - \frac{6}{14} \times \log_2\left(\frac{6}{14}\right) - \frac{4}{14} \times \log_2\left(\frac{4}{14}\right) = 1.557$$

**Gain is calculate before itself. Its value is 0.029**

- gain_ratio(income) = 0.029/1.557 = 0.019

- The attribute with the maximum gain ratio is selected as the splitting attribute

## GINI INDEX:

Gini Index, also known as Gini impurity, **calculates the amount of probability of a specific feature that is classified incorrectly when selected randomly.** If all the elements are linked with a single class then it can be called pure.

Let's perceive the criterion of the Gini Index, like the properties of entropy, the **Gini index varies between values 0 and 1**, where 0 expresses the purity of classification, i.e. All the elements belong to a specified class or only one class exists there. And 1 indicates the random distribution of elements across various classes. The value of 0.5 of the Gini Index shows an equal distribution of elements over some classes.

While designing the decision tree, the features possessing the least value of the Gini Index would get preferred.

The Gini of a dataset is:

$$\text{Gini} = 1 - (\sum p_i^2)$$ where $p_i$ is the proportion of a label.

**Example:**

| Weekend | Weather | Parents | Money | Decision |
|---|---|---|---|---|
| W1 | Sunny | Yes | Rich | Cinema |
| W2 | Sunny | No | Rich | Tennis |
| W3 | Windy | Yes | Rich | Cinema |
| W4 | Rainy | Yes | Poor | Cinema |
| W5 | Rainy | No | Rich | Stay In |
| W6 | Rainy | Yes | Poor | Cinema |
| W7 | Windy | No | Poor | Cinema |
| W8 | Windy | No | Rich | Shopping |
| W9 | Windy | Yes | Rich | Cinema |
| W10 | Sunny | No | Rich | Tennis |

Calculate the GINI index of every attribute. The attribute which is having less gini index have the maximum information.

- Compute the Gini Index for the overall collection of training examples.
- There are four possible output variables Cinema, Tennis, Stay In and Shopping.
- The data has 6 instances of Cinema, 2 instances of Tennis, 1 instance of Stay In and 1 of shopping.
- $Gini(S) = 1 - [\left(\frac{6}{10}\right)^2 + \left(\frac{2}{10}\right)^2 + \left(\frac{1}{10}\right)^2 + \left(\frac{1}{10}\right)^2] = 0.58$

- Computation of **Gini Index for Money** Attribute
- It has **two possible values of Rich (7 examples)** and **Poor (3 examples).**
- For **Money = Poor,** there are **3 examples with "Cinema".**
- $Gini(S) = 1 - [(\frac{3}{3})^2] = 0$ ✓
- For **Money = Rich,** there are **2 examples with "Tennis", 3 examples with "Cinema"** and **1 example with "Stay in", "Shopping" each**
- $Gini(S) = 1 - [(\frac{2}{7})^2 + (\frac{3}{7})^2 + (\frac{1}{7})^2 + (\frac{1}{7})^2] = 0.694$
- $Weighted\ Average(Money)$

$$= 0 * \left(\frac{3}{10}\right) + 0.694 * \left(\frac{7}{10}\right) = 0.486$$

| Weekend | Weather | Parents | Money | Decision |
|---|---|---|---|---|
| W1 | Sunny | Yes | Rich | Cinema |
| W2 | Sunny | No | Rich | Tennis |
| W3 | Windy | Yes | Rich | Cinema |
| W4 | Rainy | Yes | Poor | Cinema |
| W5 | Rainy | No | Rich | Stay In |
| W6 | Rainy | Yes | Poor | Cinema |
| W7 | Windy | No | Poor | Cinema |
| W8 | Windy | No | Rich | Shopping |
| W9 | Windy | Yes | Rich | Cinema |
| W10 | Sunny | No | Rich | Tennis |

In the same way calculate for parents and weather attribute

- Computation of **Gini Index for Parents** Attribute
- It has two possible values of **Yes (5 examples)** and **No (5 examples).**
- For **Parents = Yes,** there are **5 examples**, all with "Cinema".
- $Gini(S) = 1 - [(\frac{5}{5})^2] = 0$
- For **Parents = No,** there are **2 examples with "Tennis", 1 example with "Stay in", "Shopping" and "Cinema" each**
- $Gini(S) = 1 - [(\frac{2}{5})^2 + (\frac{1}{5})^2 + (\frac{1}{5})^2 + (\frac{1}{5})^2] = 0.72$
- $Weighted\ Average(Parents)$

$$= 0 * \left(\frac{5}{10}\right) + [0.72 * \left(\frac{5}{10}\right) = 0.36$$

- Computation of **Gini Index for Weather** Attribute
- It has three possible values of **Sunny (3 examples)**, **Rainy (3 examples)** and **Windy (4 examples).**
- For **Weather = Sunny**, there are **2 examples with "Cinema"** and **1 with "Tennis".**
- $Gini(Sunny) = 1 - [(\frac{2}{3})^2 + (\frac{1}{3})^2] = 0.444$
- For **Weather = Rainy**, there are **2 examples with "Cinema"** and **1 example with "Stay in"**
- $Gini(Rainy) = 1 - [(\frac{2}{3})^2 + (\frac{1}{3})^2] = 0.444$
- For **Weather = Windy**, there are **3 examples with "Cinema"** and **1 example with "Shopping"**
- $Gini(Windy) = 1 - [(\frac{3}{4})^2 + (\frac{1}{4})^2] = 0.375$

$$Weighted\ Average(Weather)$$

$$= 0.444 * \left(\frac{3}{10}\right) + 0.444 * \left(\frac{3}{10}\right) + 0.375 * \left(\frac{4}{10}\right)$$

$$= 0.416$$

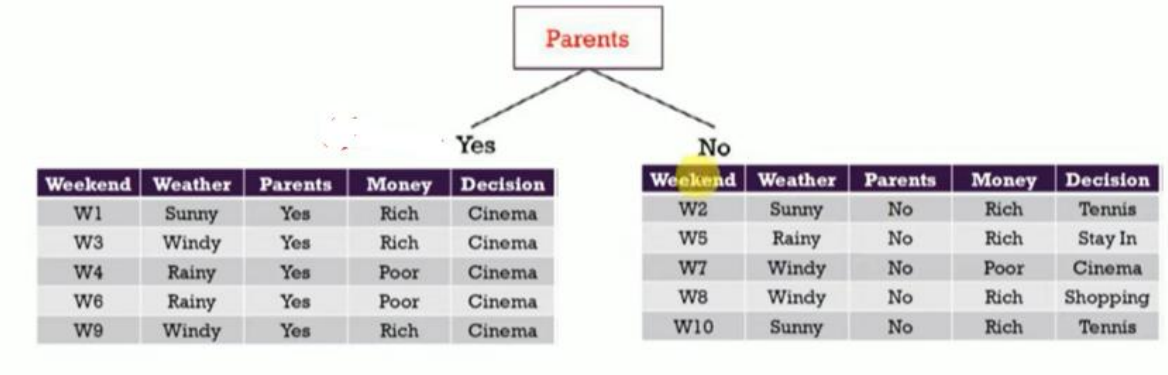
We will get

For Weather - Gini Index: 0.416

For Parents - Gini Index: 0.36

For Money - Gini Index: 0.486

In the above parents gini index is minimum , so Parents is selected as root node




| Weekend | Weather | Parents | Money | Decision |
|---|---|---|---|---|
| W1 | Sunny | Yes | Rich | Cinema |
| W3 | Windy | Yes | Rich | Cinema |
| W4 | Rainy | Yes | Poor | Cinema |
| W6 | Rainy | Yes | Poor | Cinema |
| W9 | Windy | Yes | Rich | Cinema |

| Weekend | Weather | Parents | Money | Decision |
|---|---|---|---|---|
| W2 | Sunny | No | Rich | Tennis |
| W5 | Rainy | No | Rich | Stay In |
| W7 | Windy | No | Poor | Cinema |
| W8 | Windy | No | Rich | Shopping |
| W10 | Sunny | No | Rich | Tennis |

## Issues in decision tree learning

- determining how deeply to grow the decision tree,
- handling continuous attributes,
- choosing an appropriate attribute selection measure,
- handling training data with missing attribute values,
- handling attributes with differing costs, and
- improving computational efficiency.

### A. Determining how deeply to grow the decision tree

A good machine learning model should generalize well to unseen data. Poor generalization often arises from **underfitting** or **overfitting**.

- **Underfitting**

  Occurs when the model fails to capture data patterns, often due to too little data or using overly simple models (e.g., linear model for non-linear data). It leads to low accuracy and can be reduced by using more data and proper feature selection.

  **Overfitting**

- Occurs when the model learns noise or irrelevant details, usually with overly complex models. Decision trees like **ID3** are prone to overfitting, especially with noisy or small datasets. Overfitting happens when a hypothesis fits training data better than an alternative but performs worse on unseen data.

**Avoiding Overfitting —**

There are several approaches to avoiding overfitting in decision tree learning. These can be grouped into two classes:

**- Pre-pruning (avoidance):** Stop growing the tree earlier, before it reaches the point where it perfectly classifies the training data

**- Post-pruning (recovery):** Allow the tree to overfit the data, and then post-prune the tree

**Tree Pruning:**

Pruning means to change the model by deleting the child nodes of a branch node. The pruned node is regarded as a leaf node. Leaf nodes cannot be pruned.
A decision tree consists of a root node, several branch nodes, and several leaf nodes.

- The root node represents the top of the tree. It does not have a parent node, however, it has different child nodes.
- Branch nodes are in the middle of the tree. A branch node has a parent node and several child nodes.
- Leaf nodes represent the bottom of the tree. A leaf node has a parent node. It does not have child nodes.

**There are two common approaches to tree pruning:**

**pre pruning and post pruning.**

**Pre pruning:** In the pre pruning approach, a tree is "pruned" by halting its construction early (e.g., by deciding not to further split or partition the subset of training tuples at a given node). Upon halting, the node becomes a leaf. The leaf may hold the most frequent class among the subset tuples or the probability distribution of those tuples.

When constructing a tree, measures such as statistical significance, information gain, Gini index, and so on can be used to assess the goodness of a split. If partitioning the tuples at a node would result in a split that falls below a pre specified threshold, then further partitioning of the given subset is halted. There are difficulties, however, in choosing an appropriate threshold. High thresholds could result in oversimplified trees, whereas low thresholds could result in very little simplification.

**Post Pruning:**

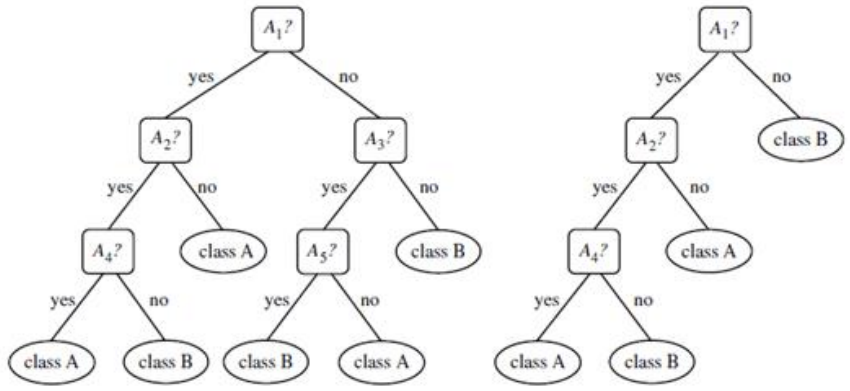



**Figure 6.6** An unpruned decision tree and a pruned version of it.

The second and more common approach is post pruning, which removes subtrees from a "fully grown" tree. A subtree at a given node is pruned by removing its branches and replacing it with a leaf. The leaf is labelled with the most frequent class among the subtree being replaced. For example, notice the subtree at node "A3?" in the un pruned tree of Figure 6.6. Suppose that the most common class within this subtree is "class B." In the pruned version of the tree, the sub tree in question is pruned by replacing it with the leaf "class B."

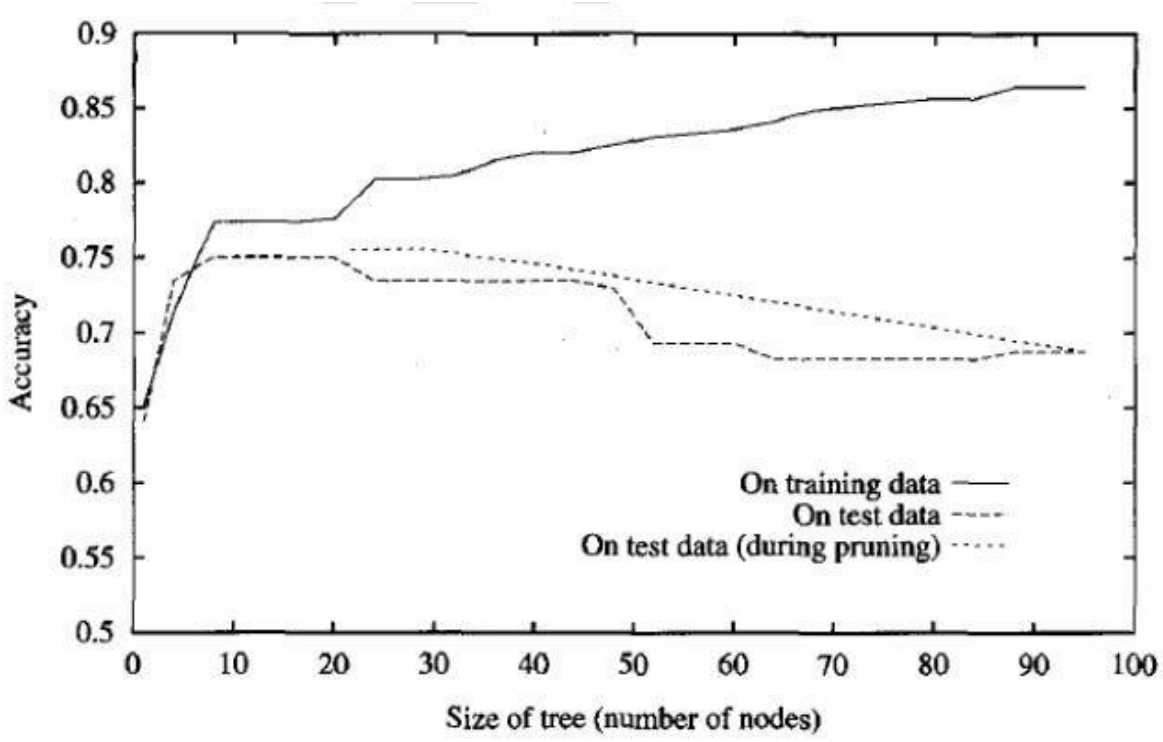
The **cost complexity pruning algorithm used in CART is an example of the post pruning approach**. This approach considers the cost complexity of a tree to be a function of the number of leaves in the tree and the error rate of the tree (where the error rate is the percentage of tuples misclassified by the tree). It starts from the bottom of the tree. For each internal node, N, it computes the cost complexity of the subtree at N, and the cost complexity of the subtree at N if it were to be pruned (i.e., replaced by a leaf node). The two values are compared. If pruning the subtree at node N would result in a smaller cost complexity, then the

subtree is pruned. Otherwise, it is kept. A pruning set of class-labeled tuples is used to estimate cost complexity. This set is independent of the training set used to build the un-pruned tree and of any test set used for accuracy estimation. The algorithm generates a set of progressively pruned trees. In general, the smallest decision tree that minimizes the cost complexity is preferred.

### 1. Reduced Error Pruning

How exactly might we use a validation set to prevent overfitting? One approach, called reduced-error pruning

- Reduced-error pruning, is to consider each of the decision nodes in the tree to be candidates for pruning
- Pruning a decision node consists of removing the subtree rooted at that node, making it a leaf node, and assigning it the most common classification of the training examples affiliated with that node
- Nodes are removed only if the resulting pruned tree performs no worse than-the original over the validation set.
- Reduced error pruning has the effect that any leaf node added due to coincidental regularities in the training set is likely to be pruned because these same coincidences are unlikely to occur in the validation set




As earlier figure accuracy vs size of the tree, the accuracy of the tree is shown measured over both training examples and test examples.

- The additional line in figure shows accuracy over the test examples as the tree is pruned. When pruning begins, the tree is at its maximum size and lowest accuracy

over the test set. As pruning proceeds, the number of nodes is reduced and accuracy over the test set increases.

- The available data has been split into three subsets: the training examples, the validation examples used for pruning the tree, and a set of test examples used to provide an unbiased estimate of accuracy over future unseen examples. The plot shows accuracy over the training and test sets.

Using a separate set of data to guide pruning is an effective approach provided a large amount of data is available. One common heuristic is: the training set constitutes 60% of all data, the validation set 20%, and the test set 20%. The major drawback of this approach is that when data is limited, withholding part of it for the validation set reduces even further the number of examples available for training.

The following section presents an alternative approach to pruning that has been found useful in many practical situations where data is limited. Many additional techniques have been proposed as well, involving partitioning the available data several different times in multiple ways, then averaging the results.

**2. Rule Post-Pruning**

Rule post-pruning involves the following steps:

- Infer the decision tree from the training set, growing the tree until the training data is fit as well as possible and allowing overfitting to occur.
- Convert the learned tree into an equivalent set of rules by creating one rule for each path from the root node to a leaf node.
- Prune (generalize) each rule by removing any preconditions that result in improving its estimated accuracy.
- Sort the pruned rules by their estimated accuracy, and consider them in this sequence when classifying subsequent instances.

To illustrate, consider again the decision tree shown in above figure. In rule post-pruning, one rule is generated for each leaf node in the tree. Each attribute test along the path from the root to the leaf becomes a rule antecedent (precondition) and the classification at the leaf node becomes the rule consequent (postcondition).

The impact of reduced-error pruning on the accuracy of the decision tree is illustrated below :

**Decision Tree Example (PlayTennis dataset)**

Suppose the learned decision tree has a path:

IF (Outlook = Sunny) AND (Humidity = High) THEN PlayTennis = No

**Step 1: Convert Tree Path → Rule**

IF (Outlook = Sunny) ^ (Humidity = High) → PlayTennis = No

**Step 2: Prune Rule Preconditions**

- Check removing (Humidity = High) → Rule becomes

IF (Outlook = Sunny) → PlayTennis = No

if accuracy **improves or remains the same**, keep it pruned.

Otherwise, check removing (Outlook = Sunny).

**Step 3: Keep the Most Accurate Rule**

Suppose pruning (Humidity = High) gives better accuracy, then final rule is:

IF (Outlook = Sunny) → PlayTennis = No

The rule is now **simpler, more general, and avoids overfitting**.

**B. Incorporating Continuous-Valued Attributes**

Our initial definition of ID3 is restricted to attributes that take on a discrete set of values.

1. The target attribute whose value is predicted by learned tree must be discrete valued.

2. The attributes tested in the decision nodes of the tree must also be discrete valued.

This second restriction can easily be removed so that continuous-valued decision attributes can be incorporated into the learned tree. For an attribute A that is continuous-valued, the algorithm can dynamically create a new boolean attribute A, that is true if A < c and false otherwise. The only question is how to select the best value for the threshold c.

**Illustration**: Suppose we wish to include the continuous-valued attribute Temperature. Suppose further that the training examples associated with a particular node in the decision tree have the following values for Temperature and the target attribute PlayTennis.

| | | | | | | |
|---|---|---|---|---|---|---|
| *Temperature:* | 40 | 48 | 60 | 72 | 80 | 90 |
| *PlayTennis:* | No | No | Yes | Yes | Yes | No |

**What threshold-based boolean attribute should be defined based on Temperature?**

- ***Pick a threshold, c, that produces the greatest information gain.*** By sorting the examples according to the continuous attribute A, then identifying adjacent examples that differ in their target classification, we can generate a set of candidate thresholds midway between the corresponding values of A. It can be shown that the value of c that maximizes information gain must always lie at such a boundary. These candidate thresholds can then be evaluated by computing the information gain associated with each.
- In the current example, there are two candidate thresholds, corresponding to the values of Temperature at which the value of PlayTennis changes: (48 + 60)/2, and (80 + 90)/2.

- The information gain can then be computed for each of the candidate attributes, Temperature >54, and Temperature >85 and the best can be selected (Temperature >54)
- This dynamically created boolean attribute can then compete with the other discrete-valued candidate attributes available for growing the decision tree.

**c. Alternative Measures for Selecting Attributes**

Information Gain often shows a bias toward attributes with many distinct values, such as *Date*, which can perfectly split training examples into tiny subsets but fails to generalize well to unseen data. To address this, Quinlan (1986) introduced the **Gain Ratio**, which adjusts the Information Gain by dividing it with **Split Information**, the entropy of the data with respect to the attribute's values. This penalizes attributes that create many uniform splits, reducing the likelihood of selecting poor predictors like *Date*. However, because the denominator can be very small (when most examples share the same value), Gain Ratio may become undefined or artificially large. To overcome this, a heuristic is often applied: first compute the Gain for each attribute, then apply the Gain Ratio only to those with above-average Gain.

**Example:** Suppose we have a **PlayTennis dataset** with the attribute *Date* (unique for each day) and a few other attributes like *Outlook* or *Humidity*.

| **Date** | **Outlook** | **Humidity** | **PlayTennis** |
|---|---|---|---|
| 1-Jan | Sunny | High | No |
| 2-Jan | Sunny | Normal | Yes |
| 3-Jan | Overcast | High | Yes |
| 4-Jan | Rain | High | No |
| … | … | … | … |

if we try to split on **Date**, each value is unique → each subset has **only one example**.

- This gives **Information Gain = maximum** (since subsets are pure).
- But the attribute *Date* does not capture any real pattern about *PlayTennis* — it only memorizes training data.
- On unseen data (new dates), this rule fails completely.

Now apply **Gain Ratio**:

- Split Information for *Date* is very high because the data is split into many tiny subsets.

- Hence,

$$GainRatio(Date) = \frac{Gain(Date)}{SplitInformation(Date)} \approx 0$$

- So Date will not be chosen as the splitting attribute.
- In contrast, attributes like Outlook (Sunny/Overcast/Rain) split examples into larger, meaningful groups, giving higher Gain Ratio and better generalization.

**D. Handling Missing Attribute Values**

In certain cases, the available data may be missing values for some attributes. For example, in a medical domain in which we wish to predict patient outcome based on various laboratory tests, it may be that the Blood-Test-Result is available only for a subset of the patients. In such cases, it is common to estimate the missing attribute value based on other examples for which this attribute has a known value.

**E. Handling Attributes with Differing Costs**

In some learning tasks the instance attributes may have associated costs. For example, in learning to classify medical diseases we might describe patients in terms of attributes such as Temperature, BiopsyResult, Pulse, BloodTestResults, etc. These attributes vary significantly in their costs, both in terms of monetary cost and cost to patient comfort.

In such tasks, we would prefer decision trees that use low-cost attributes where possible, relying on high-cost attributes only when needed to produce reliable classifications.

Instance-Based Learning- Introduction, k-nearest neighbors' algorithm, remarks on lazy and eager learning

**LAZY Learners ( or Learning from your Neighbors):** Imagine a contrasting lazy approach, in which the learner instead waits until the last minute before doing any model construction in order to classify a given test tuple. That is, when given a training tuple, a lazy learner simply stores it (or does only a little minor processing) and waits until it is given a test tuple. Only when it sees the test tuple does it perform generalization in order to classify the tuple based on its similarity to the stored training tuples.

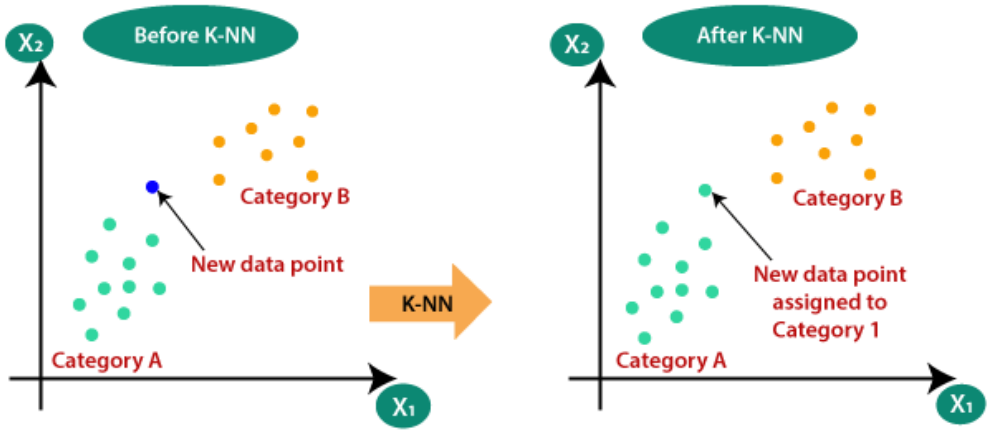
## A. K-Nearest Neighbor(KNN) Algorithm for Machine Learning

- K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on Supervised Learning technique.
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.
- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.
- **Example:** Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.



## Why do we need a K-NN Algorithm?

Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x1, so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:




## How does K-NN work?

The K-NN working can be explained on the basis of the below algorithm:

**Step-1:** Select the number K of the neighbors

**Step-2:** Calculate the Euclidean distance of **K number of neighbors**

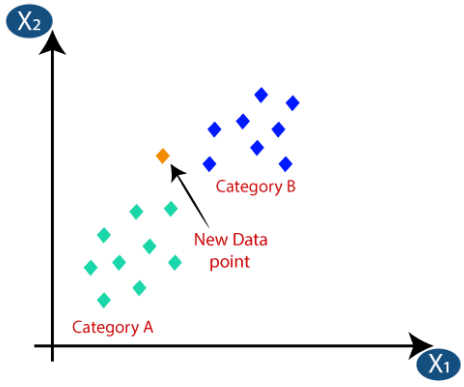
**Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.

**Step-4:** Among these k neighbors, count the number of the data points in each category.

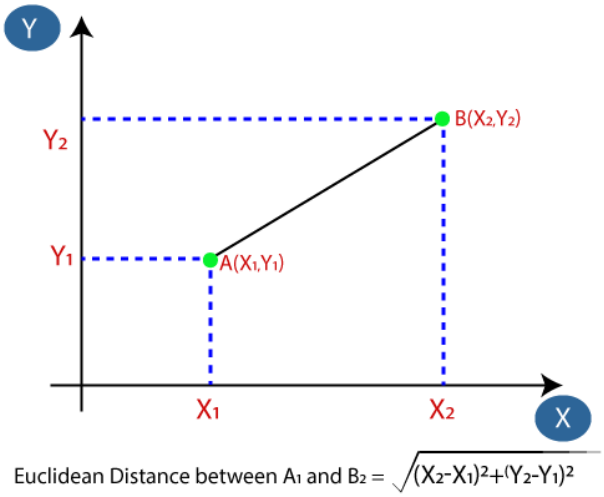
**Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.

**Step-6:** Our model is ready.

Suppose we have a new data point and we need to put it in the required category. Consider the below image:



- Firstly, we will choose the number of neighbors, so we will choose the k=5.
- Next, we will calculate the **Euclidean distance** between the data points. The Euclidean distance is the distance between two points, which we have already studied in geometry. It can be calculated as:




- By calculating the Euclidean distance we got the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B. Consider the below image:




- As we can see the 3 nearest neighbors are from category A, hence this new data point must belong to category A.

## How to select the value of K in the K-NN Algorithm?

Below are some points to remember while selecting the value of K in the K-NN algorithm:

- There is no particular way to determine the best value for "K", so we need to try some values to find the best out of them. The most preferred value for K is 5.

- A very low value for K such as K=1 or K=2, can be noisy and lead to the effects of outliers in the model.
- Large values for K are good, but it may find some difficulties.

**Advantages of KNN Algorithm:**

- It is simple to implement.
- It is robust to the noisy training data
- It can be more effective if the training data is large.

**Disadvantages of KNN Algorithm:**

- Always needs to determine the value of K which may be complex some time.
- The computation cost is high because of calculating the distance between the data points for all the training samples.

- Ex:

| Sepal Length | Sepal Width | Species |
|---|---|---|
| 5.3 | 3.7 | Setosa |
| 5.1 | 3.8 | Setosa |
| 7.2 | 3.0 | Virginica |
| 5.4 | 3.4 | Setosa |
| 5.1 | 3.3 | Setosa |
| 5.4 | 3.9 | Setosa |
| 7.4 | 2.8 | Virginica |
| 6.1 | 2.8 | Verscicolor |
| 7.3 | 2.9 | Virginica |
| 6.0 | 2.7 | Verscicolor |
| 5.8 | 2.8 | Virginica |
| 6.3 | 2.3 | Verscicolor |
| 5.1 | 2.5 | Verscicolor |
| 6.3 | 2.5 | Verscicolor |
| 5.5 | 2.4 | Verscicolor |

- New instance is

| Sepal Length | Sepal Width | Species |
|---|---|---|
| 5.2 | 3.1 | ? |

- Step 1: Find distance

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(x-a)^2 + (y-b)^2}$$

- x- new instance (sepal length)
- y- new instance (sepal width)
- a- old instance (sepal length)
- b- old instance (sepal width)

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(5.2-5.3)^2 + (3.1-3.7)^2}$$

$$\text{Distance (Sepal Length, Sepal Width)} = 0.608$$

| Sepal Length | Sepal Width | Species | Distance |
|---|---|---|---|
| 5.3 | 3.7 | Setosa | 0.608 |
| 5.1 | 3.8 | Setosa | 0.707 |
| 7.2 | 3.0 | Virginica | 2.002 |
| 5.4 | 3.4 | Setosa | 0.36 |
| 5.1 | 3.3 | Setosa | 0.22 |
| 5.4 | 3.9 | Setosa | 0.82 |
| 7.4 | 2.8 | Virginica | 2.22 |
| 6.1 | 2.8 | Verscicolor | 0.94 |
| 7.3 | 2.9 | Virginica | 2.1 |
| 6.0 | 2.7 | Verscicolor | 0.89 |
| 5.8 | 2.8 | Virginica | 0.67 |
| 6.3 | 2.3 | Verscicolor | 1.36 |
| 5.1 | 2.5 | Verscicolor | 0.60 |
| 6.3 | 2.5 | Verscicolor | 1.25 |
| 5.5 | 2.4 | Verscicolor | 0.75 |

Step 2: Find Rank

Which is having lowest distance rank it as 1 , next 2 and so on

| Sepal Length | Sepal Width | Species | Distance | Rank |
|---|---|---|---|---|
| 5.3 | 3.7 | Setosa | 0.608 | 3 |
| 5.1 | 3.8 | Setosa | 0.707 | 6 |
| 7.2 | 3.0 | Virginica | 2.002 | 13 |
| 5.4 | 3.4 | Setosa | 0.36 | 2 |
| 5.1 | 3.3 | Setosa | 0.22 | 1 |
| 5.4 | 3.9 | Setosa | 0.82 | 8 |
| 7.4 | 2.8 | Virginica | 2.22 | 15 |
| 6.1 | 2.8 | Verscicolor | 0.94 | 10 |
| 7.3 | 2.9 | Virginica | 2.1 | 14 |
| 6.0 | 2.7 | Verscicolor | 0.89 | 9 |
| 5.8 | 2.8 | Virginica | 0.67 | 5 |
| 6.3 | 2.3 | Verscicolor | 1.36 | 12 |
| 5.1 | 2.5 | Verscicolor | 0.60 | 4 |
| 6.3 | 2.5 | Verscicolor | 1.25 | 11 |
| 5.5 | 2.4 | Verscicolor | 0.75 | 7 |

**Step 3: Find Nearest Neighbour**

**In this example 1st ranked one is setosa (k=1 is setosa)**

**2nd ranked is setosa**

**3 rd ranked is setosa**

**4th ranked is versicolor**

**5th ranked is verginica**

**If k=5 then from the above example out of 5, 3 are setosa so new instance belongs to setosa**

## Lazy Learner vs Eager Learner

| Lazy Learning | Eager Learning |
|---|---|
| The model is built during prediction. | The model is built before prediction. |
| Relies heavily on the training data during prediction. | Less dependent on training data during prediction. |
| Faster during training, but slower during prediction due to real-time model building. | Slower during training, but faster during prediction due to pre-built model. |
| Example: k-Nearest Neighbors (KNN) | Example: Decision Trees, Support Vector Machines (SVM), Neural Networks |
| Less memory usage during training, but more during prediction. | More memory usage during training, but less during prediction. |

Example: 2

| BMI | Age | Sugar |
|---|---|---|
| 33.6 | 50 | 1 |
| 26.6 | 30 | 0 |
| 23.4 | 40 | 0 |
| 43.1 | 67 | 0 |
| 35.3 | 23 | 1 |
| 35.9 | 67 | 1 |
| 36.7 | 45 | 1 |
| 25.7 | 46 | 0 |
| 23.3 | 29 | 0 |
| 31 | 56 | 1 |

- **Apply** K nearest neighbor classifier to predict the diabetic patient with the given features BMI, Age. If the training examples are,
- Assume K=3,
- Test Example BMI=43.6, Age=40, Sugar=?

| BMI | Age | Sugar | Distance | |
|---|---|---|---|---|
| 33.6 | 50 | 1 | $\sqrt{(43.6-33.6)^2+(40-50)^2}$ | 14.14 |
| 26.6 | 30 | 0 | $\sqrt{(43.6-26.6)^2+(40-30)^2}$ | 19.72 |
| 23.4 | 40 | 0 | $\sqrt{(43.6-23.4)^2+(40-40)^2}$ | 20.20 |
| 43.1 | 67 | 0 | $\sqrt{(43.6-43.1)^2+(40-67)^2}$ | 27.00 |
| 35.3 | 23 | 1 | $\sqrt{(43.6-35.3)^2+(40-23)^2}$ | 18.92 |
| 35.9 | 67 | 1 | $\sqrt{(43.6-35.9)^2+(40-67)^2}$ | 28.08 |
| 36.7 | 45 | 1 | $\sqrt{(43.6-36.7)^2+(40-45)^2}$ | 8.52 |
| 25.7 | 46 | 0 | $\sqrt{(43.6-25.7)^2+(40-46)^2}$ | 18.88 |
| 23.3 | 29 | 0 | $\sqrt{(43.6-23.3)^2+(40-29)^2}$ | 23.09 |
| 31 | 56 | 1 | $\sqrt{(43.6-31)^2+(40-56)^2}$ | 20.37 |

| BMI | Age | Sugar | Distance | Rank |
|---|---|---|---|---|
| 33.6 | 50 | 1 | 14.14 | 2 |
| 26.6 | 30 | 0 | 19.72 | |
| 23.4 | 40 | 0 | 20.20 | |
| 43.1 | 67 | 0 | 27.00 | |
| 35.3 | 23 | 1 | 18.92 | |
| 35.9 | 67 | 1 | 28.08 | |
| 36.7 | 45 | 1 | 8.52 | 1 |
| 25.7 | 46 | 0 | 18.88 | 3 |
| 23.3 | 29 | 0 | 23.09 | |
| 31 | 56 | 1 | 20.37 | |

**Test Example**

**BMI=43.6, Age=40, Sugar=?**

**Sugar = 1**

**Example 3:**

| | Pepper | Ginger | Chilly | Liked |
|---|---|---|---|---|
| A | True | True | True | False |
| B | True | False | False | True |
| C | False | True | True | False |
| D | False | True | False | True |
| E | True | False | False | True |

- The "Restaurant A" sells burger with optional flavors: Pepper, Ginger and Chilly.
- Every day this week you have tried a burger (A to E) and kept a record of which you liked.
- Using Hamming distance, show how the 3NN classifier with majority voting would classify

**{ pepper: false, ginger: true, chilly : true}**

| | Pepper | Ginger | Chilly | Liked |
|---|---|---|---|---|
| A | True | True | True | False |
| B | True | False | False | True |
| C | False | True | True | False |
| D | False | True | False | True |
| E | True | False | False | True |

**New Example - Q: pepper: false, ginger: true, chilly : true**

| | Pepper | Ginger | Chilly | Liked | Distance |
|---|---|---|---|---|---|
| A | True | True | True | False | 1 + 0 + 0 = 1 |
| B | True | False | False | True | 1 + 1 + 1 = 3 |
| C | False | True | True | False | 0 + 0 + 0 = 0 |
| D | False | True | False | True | 0 + 0 + 1 = 1 |
| E | True | False | False | True | 1 + 1 + 1 = 3 |

**New Example - Q: pepper: false, ginger: true, chilly : true**

**Use Hamming Distance and**

**find the distance from Query Example (Q) to training examples (A-E)**

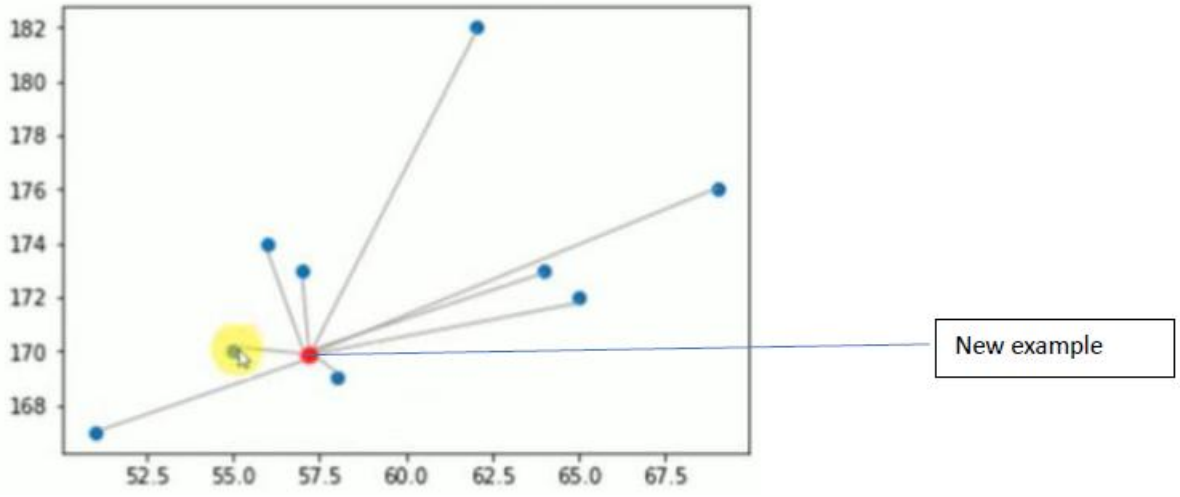
| | Pepper | Ginger | Chilly | Liked | Distance | 3NN |
|---|---|---|---|---|---|---|
| A | True | True | True | False | 1 + 0 + 0 = 1 | 2 |
| B | True | False | False | True | 1 + 1 + 1 = 3 | |
| C | False | True | True | False | 0 + 0 + 0 = 0 | 1 |
| D | False | True | False | True | 0 + 0 + 1 = 1 | 2 |
| E | True | False | False | True | 1 + 1 + 1 = 3 | |

**New Example - Q: pepper: false, ginger: true, chilly : true**

Final answer for new example is **false**

**Example 4:**

| Height (CM) | Weight (KG) | Class |
|---|---|---|
| 167 | 51 | Underweight |
| 182 | 62 | Normal |
| 176 | 69 | Normal |
| 173 | 64 | Normal |
| 172 | 65 | Normal |
| 174 | 56 | Underweight |
| 169 | 58 | Normal |
| 173 | 57 | Normal |
| 170 | 55 | Normal |
| 170 | 57 | ? |



| Height (CM) | Weight (KG) | Class | Distance | Rank |
|---|---|---|---|---|
| 169 | 58 | Normal | 1.4 | 1 |
| 170 | 55 | Normal | 2 | 2 |
| 173 | 57 | Normal | 3 | 3 |
| 174 | 56 | Underweight | 4.1 | 4 |
| 167 | 51 | Underweight | 6.7 | 5 |
| 173 | 64 | Normal | 7.6 | 6 |
| 172 | 65 | Normal | 8.2 | 7 |
| 182 | 62 | Normal | 13 | 8 |
| 176 | 69 | Normal | 13.4 | 9 |
| 170 | 57 | ? | | |